Portfolio Architecture & Interview Guide

Comprehensive Documentation for Mohd Anas's Full-Stack React 19 Portfolio

1. Languages & Core Technologies

- **TypeScript (v5.8):** Primary language across frontend and backend server functions for absolute end-to-end type safety.
- **JavaScript (ES Modules):** Modern ES2022+ features, asynchronous handling (async/await), and module bundling.
- **HTML5 & Modern CSS / Tailwind CSS v4:** Semantic markup, responsive layouts, CSS custom properties with OKLCH color space, glassmorphism, and custom keyframe animations.
- **SQL:** PostgreSQL database schema with Row-Level Security (RLS) policies for secure contact message storage.

2. Complete Tech Stack Breakdown

Frontend Architecture

- **Framework:** React 19 (v19.2) with TanStack Start (v1.x) & TanStack Router. Uses full-stack SSR/SSG with type-safe file-based routing instead of legacy SPAs.
- **Build Tool:** Vite 8 with Lightning CSS bundling for ultra-fast development and build times.
- **Styling & UI:** Tailwind CSS v4 (CSS-first engine), Radix UI Primitives (accessible headless components), Lucide React icons, Sonner toast notifications, and React Markdown for AI responses.

Backend & Serverless Layer

- **Server Runtime:** Full-stack RPC via TanStack Start `createServerFn` (executing in edge/worker environments).
- **Data Validation:** Zod (v3.24) schema validation on server functions to sanitize payloads before processing.
- **Database & BaaS:** Lovable Cloud (PostgreSQL via Supabase) featuring rigorous RLS policies for contact message persistence.
- **Third-Party Integrations:** EmailJS API for real-time dispatch to `mohd.anas.gt3@gmail.com` and Lovable AI Gateway for "Anas AI" RAG context & CS queries.

3. Top Interview Questions & How to Answer Them

Q1: Walk me through the architecture of your portfolio website.
"Built on React 19 and TanStack Start, providing full-stack capabilities with type-safe edge-ready server functions (createServerFn) validated with Zod. Uses PostgreSQL with RLS for contact messages, triggers EmailJS for immediate notifications, and runs 'Anas AI' via server-side gateway calls for RAG and coding queries."

Q2: Why choose TanStack Start & Router over Next.js?
"Offers first-class end-to-end type safety from route definition and search params to loaders and server functions. Unlike Next.js abstractions, createServerFn provides standard TypeScript RPC with zero boilerplate and pairs cleanly with Vite 8 and edge environments."

Q3: How does the contact form work securely?
"Payloads go to `submitContactMessage` where Zod sanitizes inputs. Validated messages persist in PostgreSQL via RLS while EmailJS dispatches inbox alerts concurrently, supported by client/server boundary validations and isolated try/catch error handling."

Q4: How is the 'Anas AI' chatbot implemented?
"A server-side handler keeps API keys off the client. Inputs are Zod-validated and injected with a structured system prompt containing background context. Responses are rendered via `react-markdown` supporting dual personas for recruiter inquiries and CS coding help."